UCS420: Cognitive Computing

Assignment 2 — Python Data Structures
Solved Python Programs

Q1. Lists

Using roll number **1024170237**. The digits are multiplied by 10 to form L.

```
roll_no = 1024170237

# Extract individual digits and multiply each by 10
L = [int(digit) * 10 for digit in str(roll_no)]

print("i. L =", L)

# ii. append() adds 55 at the end of the list
L.append(55)
print("After append(55):", L)

# insert() adds 65 at index 2 and shifts later elements right
L.insert(2, 65)
print("After insert(2, 65):", L)

# iii. remove() removes the first occurrence of 55 by value
L.remove(55)
print("After remove(55):", L)

# pop() removes the element at index 2
L.pop(2)
print("After pop(2):", L)

# iv. Sort ascending, then descending
L.sort()
print("Ascending:", L)

L.sort(reverse=True)
print("Descending:", L)

# v. Slicing
print("First three:", L[:3])
print("Last three:", L[-3:])

# vi. Elements greater than the average
average = sum(L) / len(L)
above_average = [x for x in L if x > average]
print("Average:", average)
print("Above average:", above_average)
```

Q2. Tuples

```
# First 8 values from the original digit list
scores = tuple([int(digit) * 10 for digit in str(roll_no)][:8])
print("scores =", scores)

# i. Highest score and its index; lowest score and frequency
highest = max(scores)
lowest = min(scores)

print("Highest score:", highest)
print("Index of highest:", scores.index(highest))
print("Lowest score:", lowest)
print("Times lowest appears:", scores.count(lowest))

# ii. Convert reversed tuple into a list
# Tuples are immutable, so they cannot be reversed in place.
reversed_scores = list(reversed(scores))
print("Reversed tuple as list:", reversed_scores)

# iii. Search for a user-entered score
user_score = int(input("Enter a score: "))
```

```
if user_score in scores:
    print("First occurrence index:", scores.index(user_score))
else:
    print("Score not present in tuple.")

# iv. Attempt to change an element
# This raises TypeError because tuple elements cannot be modified.
scores[0] = 100

# v. Tuple unpacking using *
first, second, *remaining = scores
print("First:", first)
print("Second:", second)
print("Remaining:", remaining)
```

Note: Q2(iv) intentionally raises a TypeError. Run it in a separate cell if you want the error displayed without stopping the remaining code.

Q3. Random Numbers

```
import random
from collections import Counter

# Use the roll number as the seed for reproducible results
random.seed(1024170237)

# i. Generate 100 random integers from 100 to 900 inclusive
numbers = [random.randint(100, 900) for _ in range(100)]
print("Random numbers:", numbers)

# ii. Odd numbers
odd_numbers = [n for n in numbers if n % 2 != 0]
print("Odd count:", len(odd_numbers))
print("Odd numbers:", odd_numbers)

# iii. Even numbers
even_numbers = [n for n in numbers if n % 2 == 0]
print("Even count:", len(even_numbers))
print("Even numbers:", even_numbers)

# iv. Prime number check
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

prime_numbers = [n for n in numbers if is_prime(n)]
print("Prime count:", len(prime_numbers))
print("Prime numbers:", prime_numbers)

# v. Most frequent number
frequency = Counter(numbers)
most_frequent, frequency_count = frequency.most_common(1)[0]

print("Most frequent number:", most_frequent)
print("Frequency:", frequency_count)
```

Q4. Sets

```
# Use the same 10 digits from the roll number before multiplying
digits = [int(digit) for digit in str(roll_no)]

A = {digit * 7 for digit in digits}
B = {digit * 9 for digit in digits}

print("A =", A)
print("B =", B)

# vi. Union
print("Union:", A.union(B))
```

```
# vii. Intersection
print("Intersection:", A.intersection(B))

# viii. Separate differences
print("A - B:", A.difference(B))
print("B - A:", B.difference(A))

# difference() gives values present in one set but not the other;
# symmetric_difference() gives values present in either set, but not both.

# ix. Symmetric difference
print("Symmetric difference:", A.symmetric_difference(B))

# x. Subset and superset checks
print("A subset of B:", A.issubset(B))
print("B superset of A:", B.issuperset(A))

# xi. discard() safely removes X if it exists; otherwise no error occurs.
X = int(input("Enter a value X to remove from A: "))
A.discard(X)
print("A after discard:", A)
```

Q5. Dictionaries

```
my_dict = {
    "name": "Tamanna Chadha",
    "roll_no": "1024170237",
    "branch": "Cognitive Computing",
    "age": 18,
    "city": "Patiala"
}

print("Original dictionary:", my_dict)

# i. Rename city to location without manually recreating the dictionary
my_dict["location"] = my_dict.pop("city")
print("After renaming city:", my_dict)

# ii. Add CGPA
my_dict["cgpa"] = 8.5
print("After adding CGPA:", my_dict)

# iii. Increase age by 1
my_dict["age"] += 1
print("After increasing age:", my_dict)

# iv. Delete branch using pop() and del on separate copies
dict_pop = my_dict.copy()
removed_branch = dict_pop.pop("branch")
print("Using pop():", dict_pop)
print("Returned value:", removed_branch)

dict_del = my_dict.copy()
del dict_del["branch"]
print("Using del:", dict_del)

# pop() removes and returns the value; del removes without returning it.

# v. Iterate using items()
for key, value in my_dict.items():
    print(f"{key} → {value}")

# vi. Safe check for a missing email key
if "email" in my_dict:
    print("Email:", my_dict["email"])
else:
    print("Email key is not present.")

# vii. Friend dictionary with fictional details
friend_dict = {
    "name": "Riya Sharma",
    "roll_no": "1024170238",
    "branch": "Cognitive Computing",
    "age": 19,
    "city": "Chandigarh"
}

merged_dict = {**my_dict, **friend_dict}
print("Merged dictionary:", merged_dict)

# When both dictionaries have the same key, values from friend_dict win
# because it is unpacked second.

# viii. Keep only key-value pairs whose values are strings
string_values = {
    key: value for key, value in my_dict.items()
    if isinstance(value, str)
}
print("Only string values:", string_values)
```

Important

Replace the example CGPA, age, branch, and any other personal/academic details with your actual values before submission if they differ. The roll number used above is 1024170237.